FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 10. Funcionamiento interno de Git

Comandos de fontanería, objetos, referencias, protocolos y mantenimiento

Entender cómo almacena Git la información explica por qué es tan rápido y robusto. Este capítulo recorre su estructura interna.

10.1 Fontanería y porcelana

Los comandos de **porcelana** son los amigables de uso diario (`add`, `commit`, `log`); los de **fontanería** (*plumbing*) son de bajo nivel y trabajan directamente con la base de datos (`hash-object`, `cat-file`, `update-index`, `write-tree`, `commit-tree`). El directorio `.git` contiene `HEAD`, `index`, `config`, `objects/` y `refs/`.

10.2 Objetos de Git

Git es un sistema de archivos direccionable por contenido: cada objeto se guarda con una clave que es el hash de su contenido. Existen cuatro tipos:

- **blob:** el contenido de un archivo.
- **tree:** un directorio; lista nombres, permisos y hashes de blobs u otros árboles.
- **commit:** apunta a un árbol, a sus commits padre y guarda autor, fecha y mensaje.
- **tag (anotada):** apunta a un objeto con un mensaje y autor.

```
echo "contenido" | git hash-object -w --stdin
git cat-file -p <hash>
git cat-file -t <hash>     # tipo de objeto
```

10.3 Referencias

Una referencia es un archivo que contiene un hash, para no tener que recordarlo. Las ramas viven en `.git/refs/heads/`, las etiquetas en `.git/refs/tags/` y los remotos en `.git/refs/remotes/`. `HEAD` es una referencia simbólica que apunta a la rama actual.

```
cat .git/HEAD
git update-ref refs/heads/prueba <hash>
git symbolic-ref HEAD
```

10.4 Archivos empaquetados (packfiles)

Para ahorrar espacio, Git comprime los objetos en archivos *pack* que guardan versiones parecidas como diferencias (deltas). `git gc` hace esta compactación automáticamente.

10.5 Refspec y protocolos de transferencia

Un *refspec* indica cómo se relacionan las ramas remotas y locales, con formato `+origen:destino` (por ejemplo `+refs/heads/*:refs/remotes/origin/*`). Para transferir datos, Git usa el protocolo *tonto* (HTTP simple, solo descarga archivos) o el *inteligente* (procesos `send-pack`/`receive-pack` y `upload-pack`/`fetch-pack`), que negocia qué falta y envía solo eso.

10.6 Mantenimiento y recuperación de datos

Con el tiempo se acumulan objetos sin referencia. Git los mantiene un tiempo antes de eliminarlos, lo que permite recuperar trabajo.

```
git gc
git fsck --full          # verifica integridad y objetos huérfanos
git prune
git reflog
```

10.7 Variables de entorno

El comportamiento de Git puede ajustarse con variables como `GIT_DIR`, `GIT_WORK_TREE`, `GIT_AUTHOR_NAME`, `GIT_COMMITTER_EMAIL`, `GIT_EDITOR` y `GIT_TRACE` (útil para depurar).

Conclusión del capítulo

Conocer los objetos y referencias permite entender por qué las ramas son baratas, por qué el historial es difícil de corromper y cómo recuperar información que parecía perdida.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2